

# Dendritic actin growth near the membrane

Current simulation scheme vs. the dendritic-nucleation model — analysis & recommendations

BoxOfActin · branch `membrane-render-and-dense-v5` · 2026-06-15 · sources: `FilSegment.java`, `Arp23.java`, `StickyNode.java`, `Env.java`; `JOURNAL.md` 2026-06-14/15; `MEMBRANE_MODEL_NOTES.md`

---

## Part 1 — What the code actually does today

---

The "dendritic branching kit" (commits `58b35ae`, `9f7ad66`) is an exploratory, default-off layer on top of the existing Arp2/3 machinery. The mechanism, end to end:

**Branch nucleation** (`FilSegment.checkBranching()`, ~line 1204). A filament's barbed end (`end2`) becomes branch-eligible if either:

- it is near a "hot Arp activator" mesh node (`end2NearArpFactor`), **or**
- (new) its barbed end sits within `branchMembraneDist` below a **fixed  $z = 0$  plane** (`getEnd2Z() > -memDist`).

If eligible, it fires with probability `branchRateNearArpFactors` · `arpConc` · `biochemDeltaT` per biochem tick, placing a daughter at a uniformly random spot within `branchZone` (50 nm) of the barbed end. The daughter is nucleated at a hard-coded **70°** (`arp23AlphaAngle`) at a random azimuth around the mother — geometrically correct Arp2/3.

**Membrane localization** is currently the  $z$ -plane rule above. The biologically-shaped alternative —

`StickyNode.biochemStep()`'s `iAmHotRho` "Rho-hot patch" that nucleates filaments at the surface — **exists but is hard-disabled** (`getHotProb = 0`, `hotSpotSpreadProb = 0`, marked "testing only").

**Elongation** (`addMonomerSim`):  $\leq 1$  monomer per biochem fire, so growth velocity  $\approx 1$  monomer / `biochemDeltaT`, saturated and nearly independent of `kATP0n2 / actinConc`. Monomer is drawn from a **single global scalar** `Env.actinConc` (15  $\mu\text{M}$ ), decremented/incremented via `Crucible.takeMonomer/putMonomer`.

**Capping** (`checkCapping()`): two regimes.

- Default (`membraneCapDist = 0`): genuine stochastic capping, `capRate` · `capConc` · `biochemDeltaT`, from the scalar `capConc` (2  $\mu\text{M}$ ).
- Lamellipodium regime (`membraneCapDist > 0`): **deterministic, geometry-gated** — `end2Capped = (end2TipC ≥ membraneCapDist)`. Uncapped at the cortex, force-capped everywhere else.

**Force coupling**: `stericPolyFactor` softly attenuates poly rate when the tip is within ~one monomer of an obstacle; `membraneFaceCollideOn` does one-sided point-vs-triangle collision against the membrane.

The "**three throttles**" (journal 2026-06-15): branch density = `arpConc`; filament length = `biochemDeltaT`; cortex confinement = `membraneCapDist` — tuned to be near-orthogonal.

The journal is admirably honest about the hacks. The two worth building on: the  $z = 0$  plane is not the real (bulging) membrane, and `stericPolyFactor` is "ad-hoc, not a derived force-velocity law."

## Part 2 — Comparison to the dendritic-nucleation model

The reference biology is the Pollard–Borisý dendritic nucleation / array-treadmilling model (Pollard & Borisý 2003; Svitkina & Borisý 1999; Mullins et al. 1998). Scoring the code against it:

| Mechanism                                             | Biology                                                                                                          | Code today                                                                                                         | Verdict                                               |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Branch angle<br>70°, random<br>azimuth                | Arp2/3 binds mother side,<br>daughter at $70^\circ \pm 7^\circ$                                                  | Exactly this<br>( <code>arp23AlphaAngle</code> )                                                                   | ✓ <b>Faithful</b>                                     |
| <b>What localizes<br/>branching</b>                   | Membrane-bound NPF<br>(WAVE/WASP) activated<br>by Rac/Cdc42/PIP2 —<br>branching where the<br><i>activator</i> is | Fixed $z = 0$ geometric plane,<br>or disabled hot-Rho                                                              | ⚠ <b>Right idea, wrong mechanism</b>                  |
| <b>Which filament<br/>gets branched</b>               | Preferentially <b>newly-<br/>polymerized ATP/ADP-Pi</b><br>filament near the front<br>(Ichetovkin 2002)          | Any eligible barbed-end zone;<br><b>no nucleotide gate</b><br>( <code>notADPFraction()</code> )<br>exists, unused) | ✗ <b>Missing autocatalytic bias</b>                   |
| <b>Debranching /<br/>Arp2/3<br/>turnover</b>          | Branches dissociate —<br>ADP-Pi → ADP aging +<br>GMF debranching (Gandhi<br>2010); array treadmills              | <b>None.</b> <code>removeArp23</code> only<br>fires when the daughter fully<br>depolymerizes                       | ✗ <b>Missing — net only accretes</b>                  |
| <b>Capping</b>                                        | CP caps <b>stochastically<br/>everywhere</b> (bulk scalar);<br>spatial pattern is <i>emergent</i>                | Geometry-gated deterministic<br>cap ( <code>membraneCapDist</code> )<br>imposes the pattern                        | ⚠ <b>Right outcome, inverted mechanism</b>            |
| <b>Force–velocity</b>                                 | Elastic / Brownian ratchet,<br>$v \propto \exp(-f \cdot \delta / kT)$<br>(Mogilner–Oster)                        | <code>stericPolyFactor</code> , a<br>flat constant                                                                 | ⚠ <b>Ad-hoc proxy</b>                                 |
| Orientation<br>selection ( $\pm 35^\circ$<br>pattern) | Emergent from<br>branch+cap+treadmill<br>competition (Maly &<br>Borisý 2001)                                     | Cannot emerge — no turnover<br>to select on                                                                        | ✗ <b>Downstream of gaps above</b>                     |
| Monomer / CP<br>pools                                 | Spatially varying; leading-<br>edge depletion + funneling<br>(Akin & Mullins 2008)                               | <b>Global</b> scalars                                                                                              | ⚠ <b>OK as scalars, but global hides the gradient</b> |

**The throughline:** the code *imposes by geometry* what biology *produces by self-organization*. A real lamellipodium is thin, oriented, and treadmilling because branching is autocatalytic at the membrane, capping is bulk-stochastic, and old material debranches and depolymerizes at the rear. The code skips the loop and pins the *result* to a  $z$ -plane: branch near  $z = 0$ , cap away from  $z = 0$ . Fine as a flat-sheet snapshot — but it cannot curve, cannot treadmill, and cannot reproduce the orientation pattern.

## Part 3 — Suggested changes

---

Actin and capping protein remain **scalar fields** throughout, per the design constraint — none of this proposes explicit monomer or CP particles.

### PRIORITY 1 Localize branching by activator at the *real* surface, not a z-plane

Revive the disabled `iAmHotRho` path and make it the branch gate. Branch eligibility should be "barbed end within reach of an **active NPF-bearing membrane node**," computed from `StickyNode` proximity (`end2TipC` / mesh, already tracked), not from `getEnd2Z()`. This:

- works for a curved / closed cortex for free (keys off membrane nodes, which move);
- restores the biological knob (NPF activation density = `nucRateNearArpFactors` × an activator field), so Rac/Cdc42/PIP2 patterning becomes expressible;
- lets you delete the fixed-plane assumption the journal calls out twice.

Highest-leverage fix: replaces *both* `branchMembraneDist` and the `membraneCapDist` z-rule's rationale with the actual surface.

### PRIORITY 2 Add Arp2/3 debranching (the missing turnover term)

Without it there is no treadmilling and no steady state — the network can only thicken. Add a stochastic per-branch dissociation in `Arp23` at rate `kDebranch · biochemDeltaT`, **gated on the daughter's nucleotide age** via the `notADPFraction()` you already compute: branches on ADP filament dissociate fast, ATP/ADP-Pi branches are stable (Gandhi 2010; Mahaffy & Pollard). `removeArp23` already performs the topology change — this just calls it on a clock instead of only on full depoly. This is what lets the array recycle and what makes  $\pm 35^\circ$  orientation emergent rather than impossible.

### PRIORITY 3 Make capping a true bulk scalar process; let localization emerge

Demote `membraneCapDist` from a mechanism to (at most) a diagnostic. Cap stochastically from `capConc` everywhere (the default regime already does this). The thin membrane-hugging layer then **emerges** from P1 (branching only at the surface) + P2 (rear debranching/depoly) — more honest, and it tracks a curved membrane automatically. Keep `membraneCapDist` as a fast "force the outcome" shortcut, but stop treating it as the model.

### PRIORITY 4 Bias branching to newly-polymerized filament

Gate `checkBranching()` on local nucleotide state: weight branch probability by `notADPFraction()` near the barbed end (ATP/ADP-Pi preferred). One extra factor in the existing probability expression — turns the branch step genuinely autocatalytic (new filament begets branches begets new filament), the engine of the dendritic array.

### PRIORITY 5 Replace `stericPolyFactor` with a Brownian-ratchet law

You already have the per-tip membrane load from the face-collision reaction force. Swap the flat `stericPolyFactor` for `rate · exp(-f ·  $\delta$  / kT)` with  $\delta$  = monomer half-size (Mogilner–Oster). Also fixes the poly-through-membrane tunneling the journal warns about: load rises → rate falls smoothly to zero at the stall force, rather than a constant fraction that can outrun the one-sided push.

## PRIORITY 6 Optional, still scalar — spatially bin the actin/CP fields

A single global `actinConc` can't show leading-edge monomer depletion or the capping "funnel." Since you have a `Mesh`, the cheapest faithful upgrade is per-mesh-cell scalar fields for `actinConc` / `capConc` with diffusive relaxation between cells — still fields, no particles. The only change here that touches the monomer/CP representation, and it stays inside the "implicit scalar field" mandate. Defer until P1–P2 give you a treadmill array that *would* deplete a local pool.

## ALSO Cheap correctness items flagged in the journal

- **Select the IC by parameter, not the `xLinkTesting` / `nodeLinkTesting` compile-time flags** — the journal documents this contaminating a real run. A correctness wart, not just ergonomics.
- **Default `arpConc` to nonzero** in any "branching" param file, or assert at startup — silent dead branching has already bitten (memory + journal both record it).

**Recommended sequence.** P1 → P2 → P3 form one coherent change — *localize branching at the real surface, add turnover, let capping be bulk* — together converting the system from "geometrically-pinned snapshot" to "self-organizing treadmill array." P4–P5 sharpen the dynamics; P6 is the scalar-field refinement once there's a treadmill to feed it. Everything stays within the agent-based-filaments + scalar-field-monomers paradigm the codebase already commits to.